

APK 逆向学习

完整小白文档

12 周学习路线 · 10 个实战案例

80+ 术语辞典 · 23 个工具手册

出品:由 Kiro 协助整理

APK 逆向学习 · 零基础小白完整文档

从零开始,用 AI 做老师,12 周掌握 APK 逆向

这是一套写给**完全零基础小白**的 APK 逆向工程学习文档。所有术语都用大白话讲,所有工具都有操作指南,所有步骤都能配合 AI 完成。

这份文档适合谁?

- 想学 Android 逆向但**完全零基础**的小白
- 想用 **AI 辅助学习**,提高效率的学习者
- 想做**爬虫 / 安全研究 / CTF** 的同学
- 想系统化学习,不想零散看视频的人

不适合: - 已经有基础,想看高级技巧的大佬 - 想找"破解教程"直接套用的伸手党

文档结构

APK逆向学习/

—— README.md	← 你在这里
—— 00_从零开始_看这里.md	← 入口文档:先看这个
—— 01_知识架构_一张图看懂全貌.md	← 全景地图
—— 02_名词大辞典_小白版.md	← 术语速查
—— 03_工具手册_每个工具干什么用.md	← 工具说明书
—— 04_AI辅助学习法_怎么跟AI提问.md	← AI 使用心法
—— 05_12周学习路线_每日任务.md	← 执行计划
—— 06_实战案例集_从简单到难.md	← 10 个练手项目
—— 07_避坑指南_法律与常见问题.md	← 必看:法律 + FAQ

推荐阅读顺序

第 1 天:建立全局认知(2 小时)

- 1. 00_从零开始_看这里 — 了解学什么、为什么学
- 2. 01_知识架构_一张图看懂全貌 — 看懂技能树
- 3. 07_避坑指南_法律与常见问题 — 务必先看法律部分

第 2 天:准备学习环境(2-3 小时)

- 1. 02_名词大辞典_小白版 — 浏览一遍,有印象就行
- 2. 03_工具手册_每个工具干什么用 — 按"最小装机清单"装工具
- 3. 04_AI辅助学习法_怎么跟AI提问 — 这篇最关键

第 3 天起:开始执行

- 1. 05_12周学习路线_每日任务 — 按计划每天打卡
- 2. 06_实战案例集_从简单到难 — 每周做 1 个案例

你能获得什么？

12 周后,你将掌握:

阶段	能力
第 1-3 周	读懂 Java / Smali,会用 ADB,装好全套工具
第 4-6 周	反编译 APK、去广告、重打包,理解加密算法
第 7-9 周	Frida Hook、抓 HTTPS、绕过 SSL Pinning、分析签名
第 10-12 周	Ghidra 看 so、Unidbg 模拟执行、基础脱壳

就业方向:Android 安全工程师、爬虫工程师、CTF 选手、安全研究员

学习三大原则

1 动手 > 阅读

看 10 小时视频 < 动手 1 小时。每学一个工具,立刻找个 App 试。

2 AI 是老师,不是保姆

AI 能解答你的问题,但路要你自己走。光让 AI 生成不动手,等于没学。

3 守住法律底线

- 可以:自己的 App、开源 App、授权测试、官方靶场
- 不可以:破解商业软件、窃取数据、攻击他人系统

最小装机清单

先装这 4 个就能开始:

工具	用途	下载
Android Studio	开发环境 + 模拟器	官网
jadx-gui	APK 反编译	GitHub
apktool	APK 解包 / 重打包	官网
Python 3.10+	写脚本	官网

其他工具学到对应阶段再装,详见 [03_工具手册](#)。

配合 AI 使用的快速指南

每次提问的黄金公式

【我的情况】:零基础 / 在学第 X 周
【操作系统】:Win / Mac / Linux
【我在做什么】:具体任务
【遇到什么问题】:描述 + 报错
【我试过什么】:已经尝试过的方法

更多 Prompt 模板见 [04_AI辅助学习法](#)。

遇到问题求助渠道

1. **AI**(Kiro / ChatGPT / Claude)— 70% 的问题能解决
2. **看雪论坛** - bbs.kanxue.com — 国内最专业
3. **吾爱破解** - 52pojie.cn — 新手友好
4. **GitHub Issues** — 找工具官方仓库

法律声明

本文档仅用于**技术学习**和**安全研究**目的。

使用者须承诺: - 不得用于破解商业软件并传播 - 不得用于窃取他人数据或隐私 - 不得用于非法攻击他人系统
- 不得制作、传播外挂或恶意程序

因违法使用产生的一切后果由使用者自行承担。

详细法律条款请阅读 [07_避坑指南](#)。

开始你的逆向之旅

现在就去打开 [00_从零开始_看这里](#)

记住:每个大佬都是从不知道什么是 APK 开始的。

你一定可以。

00 | 从零开始:小白入门 APK 逆向

你现在完全不懂编程也没关系,这份文档就是写给你看的。 建议按顺序读 00 → 01 → 02 → 03 → 04 → 05,边读边动手。

一、用大白话解释:什么是"APK 逆向"?

先懂两个词

- **APK** = Android 手机上安装的 App 文件(比如微信、抖音下载下来是 `xxx.apk`)
- **逆向** = "反过来推"。开发者先写代码 → 编译打包 → 变成 APK;逆向就是拿着 APK → 反推出里面的代码和逻辑

一个生活类比

想象你吃到一块超级好吃的蛋糕(APK),你想知道配方(源代码): - **正向开发**:厨师按配方做蛋糕 - **逆向工程**:你拿着蛋糕,一口一口分析里面有什么原料、什么工艺

APK 逆向做的事情就是:**把一个 App 拆开,看它内部怎么工作的。**

二、学这个能干嘛?(给你动力)

目的	具体场景
安全研究	找 App 漏洞、给厂商提 SRC(拿奖金)
去广告/去弹窗	把讨厌的广告干掉(仅限自己用)
协议分析	写爬虫时分析 App 接口的加密参数(如签名 sign)
CTF 比赛	拿奖金、拿简历亮点
职业方向	安卓安全工程师、年薪 30-80 万
学习提升	逆向能让你对"程序如何运行"的理解上一个台阶

三、为什么用 AI 学特别香？

传统学逆向: - 看 1000 页的书,看完忘 900 页 - 遇到报错没人问,卡 3 天放弃 - 术语太多,一个概念查半天

用 AI(Kiro / ChatGPT / Claude)学: - **随时提问**:任何术语秒懂,像请了个 24 小时家教 - **代码解释**:把一段 Smali 代码丢给它,它帮你翻译成成人话 - **报错修复**:装工具报错?粘贴给 AI 直接解决 - **生成脚本**:Frida Hook 脚本让 AI 写,你只需要改参数 - **个性化学习**:不懂就追问,AI 会换种方式再讲一遍

核心心法:AI 是老师,不是保姆。你要主动问、主动动手,光让它生成你不去跑,等于没学。

四、这份文档怎么用？

文档结构

APK 逆向学习/

—— 00_从零开始_看这里.md	← 你正在看的入口
—— 01_知识架构_一张图看懂全貌.md	← 先看全景地图
—— 02_名词大辞典_小白版.md	← 遇到不懂的词就查
—— 03_工具手册_每个工具干什么用.md	← 工具说明书
—— 04_AI辅助学习法_怎么跟AI提问.md	← 跟 AI 对话的模板
—— 05_12周学习路线_每日任务.md	← 按周执行的计划
—— 06_实战案例集_从简单到难.md	← 10 个练手项目
—— 07_避坑指南_法律与常见问题.md	← 法律 + FAQ

推荐阅读顺序

第 1 天(今天): 1. 读完 00_从零开始 ← 现在这篇 2. 读 01_知识架构 建立全景认知 3. 翻一遍 02_名词大辞典,有个印象就行

第 2 天: 4. 读 03_工具手册,按清单装工具 5. 读 04_AI辅助学习法,学会跟 AI 提问

第 3 天起: 6. 按 05_12周学习路线 每天执行 7. 遇到术语查 02_辞典,遇到报错问 AI 8. 每周做一个 06_实战案例

五、学习前的准备清单

硬件

- [] 一台电脑(Windows / Mac / Linux 都行,内存建议 16G+)
- [] 一部**可以 Root 的 Android 手机**(二手 Pixel 最理想,300-800 元)
- 没有也行,可以先用模拟器(雷电 / MuMu / 夜神)

软件环境

- [] **Android Studio**(用来装模拟器,官网下载)
- [] **Python 3.10+**(写脚本用)
- [] **JDK 17**(运行 Java 工具用)
- [] **Git**(下载工具)

账号

- [] **GitHub 账号**(下载各种工具)
- [] **一个 AI 账号**(Kiro / ChatGPT / Claude / 通义千问都可)
- [] **看雪论坛账号**(bbs.kanxue.com,国内最大逆向社区)

不会装? 把"我要装 XXX,我的系统是 XXX"发给 AI,它会一步步教你。

六、学习心态:5 个必须知道的真相

1. 逆向不是"黑客技术",是"阅读技术"

90% 的时间你都在**读代码**,不是在"黑"什么。别被影视剧骗了。

2. 会卡住是常态,不是你笨

每个逆向工程师都卡过 3 天解不出一个加密算法。**卡住 = 成长信号**。

3. 动手比看书重要 100 倍

看 10 小时视频 < 动手练 1 小时。**每学一个工具,立刻找个 App 试**。

4. 英文是硬伤,但能克服

工具文档、报错信息大多是英文。用 **AI 翻译 + 沉浸式翻译插件**,门槛没那么高。

5. 守住法律底线

- 可以逆向:自己的 App、开源 App、授权测试、官方靶场
 - 不能做:破解商业软件发布、窃取他人数据、恶意攻击
-

七、下一步

现在打开 **01_知识架构_一张图看懂全貌.md**,建立全局认知。

如果你有任何问题,**直接问 AI(Kiro)**: - "我看完 00,还不太明白 APK 和普通 exe 有什么区别,能解释一下吗?" - "我电脑是 Mac M1,推荐哪个模拟器?" - "我没有安卓手机,只能用模拟器学会吗?"

开始你的逆向之旅吧!

01 | 知识架构:一张图看懂全貌

看完这篇,你就能在脑子里有一张"地图",知道自己每天在学哪一块。

一、最简模型:APK 逆向 = 拆快递

```
[APK 文件] ← 一个密封的快递箱
|
| 用工具拆开(apktool / jadx)
▼
[里面的东西]
|—— 说明书(AndroidManifest.xml)
|—— 主要货物(classes.dex = 代码)
|—— 附件(resources / assets = 图片文字)
|—— 底层零件(lib/*.so = C/C++ 编译的库)
|
| 看货物怎么工作(静态阅读 + 动态跟踪)
▼
[搞懂 App 的逻辑]
|
| 修改/Hook/复现算法
▼
[达到你的目的:去广告 / 抓包 / 还原签名]
```

二、金字塔分层:从小白到高手的 5 层

第 5 层 加固对抗 (脱壳 / 反反调试 / VMP)	← 大神级
第 4 层 Native 逆向 (so 库 / C++ / ARM)	← 进阶
第 3 层 动态分析 (Frida Hook / 抓包)	← 核心
第 2 层 静态分析 (jadx / apktool)	← 入门
第 1 层 基础知识 (Android + 编程)	← 地基

每一层是什么?一句话解释

层级	名字	一句话	对应目标
1	基础知识	学 Android 怎么运行、学看懂代码	能读懂 Java
2	静态分析	把 APK 拆开,不运行直接读它的代码	去广告、破弹窗
3	动态分析	让 App 跑起来,在运行时改它的行为	抓包、爬虫、绕过登录
4	Native 逆向	分析 C/C++ 写的 .so 库(更底层)	还原加密算法
5	加固对抗	App 被加固保护了,你得"脱壳"才能看	分析商业级 App

三、详细技能树

APK 逆向

- |
 - |—— ① 基础层(地基)
 - |—— Android 基础
 - |—— 什么是 APK、DEX、ART 虚拟机
 - |—— 四大组件(Activity / Service / Broadcast / Provider)
 - |—— AndroidManifest.xml 清单文件
 - |—— APK 签名机制(v1/v2/v3)
 - |—— 编程语言
 - |—— Java(必会读)
 - |—— Kotlin(必会读)
 - |—— Smali(读懂就行,像汇编一样)
 - |—— Python(写脚本)
 - |—— JavaScript(写 Frida Hook)
 - |—— Linux 命令
 - |—— adb / shell / grep / find
 - |—— ② 静态分析层(看代码)
 - |—— APK 解包工具
 - |—— jadx-gui ← 一键反编译,最常用
 - |—— apktool ← 能重打包
 - |—— dex2jar ← 老牌工具
 - |—— 代码审计
 - |—— 看 AndroidManifest(找入口/权限)
 - |—— 搜字符串(找关键逻辑)
 - |—— 追踪函数调用链
 - |—— 识别加密算法
 - |—— 重打包
 - |—— 改 Smali 代码
 - |—— 重新签名(apksigner)
 - |—— 安装测试
 - |—— ③ 动态分析层(跑起来抓)
 - |—— Hook 框架(最重要!)
 - |—— Frida ← 神器,必学
 - |—— Xposed/LSPosed ← 需要 Root
 - |—— Objection ← Frida 的"傻瓜版"
 - |—— 抓包工具
 - |—— Charles / Fiddler ← 图形化,抓 HTTP
 - |—— mitmproxy ← 命令行,可编程
 - |—— Wireshark ← 抓底层 TCP/UDP
 - |—— 证书处理
 - |—— 安装用户证书 / 系统证书
 - |—— 绕过 SSL Pinning(证书锁定)
 - |—— 调试
 - |—— adb logcat 看日志
 - |—— smalidea 断点调试

- └── ④ Native 层(更底层)
 - └── JNI 机制
 - └── Java 怎么调用 C/C++ 代码
 - └── 静态注册 vs 动态注册
 - └── so 分析
 - └── IDA Pro ← 收费但最强
 - └── Ghidra ← 免费开源,NSA 出品
 - └── Binary Ninja ← 界面友好
 - └── 汇编基础
 - └── ARM / ARM64 指令集
 - └── 寄存器、栈、调用约定
 - └── 模拟执行
 - └── Unidbg ← 脱机跑 so(爬虫党最爱)
- └── ⑤ 加固对抗层(高级玩家)
 - └── 识别加固类型
 - └── 360 / 梆梆 / 爱加密 / 腾讯乐固 / 顶象
 - └── 脱壳
 - └── FART ← 主动调用脱壳机
 - └── BlackDex ← 易用
 - └── FRIDA-DEXDump ← 基于 Frida
 - └── 反反调试
 - └── 绕过 ptrace 检测
 - └── 绕过 Frida 检测
 - └── 绕过 Root 检测
 - └── 高级保护
 - └── OLLVM 混淆还原
 - └── VMP 虚拟化保护
 - └── 自定义指令集还原

四、三种主流"套路"(别人都这么干)

套路 A:纯静态流派 "看派"

步骤:jadx 打开 → 搜关键字 → 读代码 → 还原逻辑 **适合:**简单 App、没加固 **缺点:**遇到混淆就废

套路 B:动态 Hook 流派 "钩派" 主流

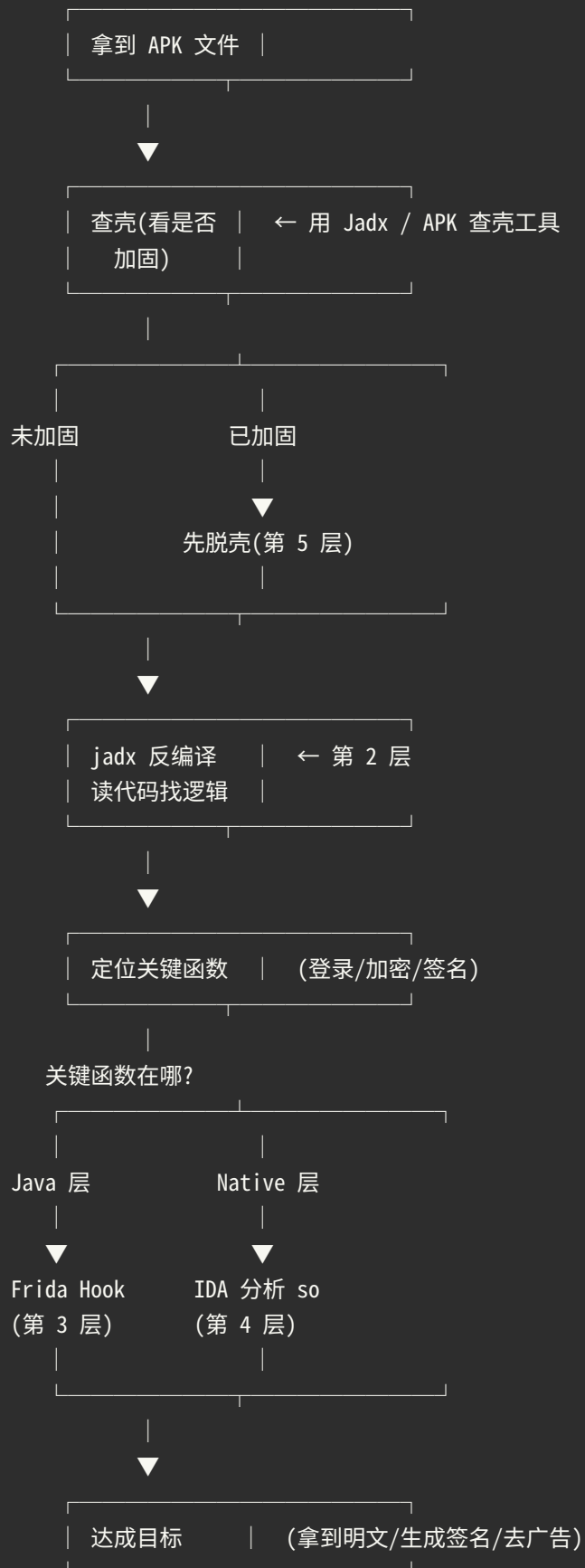
步骤:跑 App → Frida 注入 → Hook 目标函数 → 拿到明文/密钥 **适合:**90% 的日常场景 **优点:**不管代码多复杂,Hook 一下就出结果

套路 C:模拟执行流派 "仿派"

步骤:抽出 so → Unidbg 搭环境 → Python 直接调用 **适合:**批量跑签名(爬虫场景) **优点:**脱机、快

学习顺序建议:先学 A(建立阅读能力),再学 B(90% 场景能搞定),进阶学 C。

五、典型逆向流程图



六、每一层对应的"毕业标志"

层级	学完的标志(能做到这些才算过关)
① 基础	能读懂 10 行 Java 代码,能用 adb 连接手机
② 静态	能用 jadx 打开一个 App,搜字符串定位到代码
③ 动态	能用 Frida Hook 一个函数,打印它的参数和返回值
④ Native	能用 IDA 打开 .so 文件,看懂一个简单函数
⑤ 对抗	能对一个 360 加固的 App 脱壳出原始 DEX

七、不同目标的最短路径

我只想抓 App 的接口(写爬虫)

→ ① 基础 → ③ 动态(Frida + 抓包)→ 约 2 个月

我想去 App 广告

→ ① 基础 → ② 静态(会改 Smali)→ 约 1 个月

我想做安卓安全工程师

→ ① → ② → ③ → ④ → ⑤ 全部,约 6-12 个月

我想打 CTF 比赛

→ ① → ② → ④(偏重 Native 题)→ 约 3-6 个月

下一步

接下来打开 [02_名词大辞典_小白版.md](#),把上面看到的每一个陌生词都弄懂。

02 | 名词大辞典(小白版)

遇到不懂的词 → 按 **Ctrl+F** 搜索 → 看完回去继续学 所有解释都用**大白话 + 生活类比**,不讲学术定义

按首字母索引

A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | R | S | T | U | V | X | 中文

A

ADB (Android Debug Bridge)

一句话:电脑和安卓手机"对话"的工具。 **类比:**像是手机的"遥控器",你在电脑上敲命令,手机上执行。 **常用命令:** - `adb devices` 看连了哪些设备 - `adb install app.apk` 装 App - `adb shell` 进入手机的命令行 - `adb logcat` 看 App 的运行日志

AES

一句话:一种**对称加密算法**(加密和解密用同一把钥匙)。 **类比:**你和朋友约好同一个暗号,用它加密信件,也用它解开。 **逆向场景:**很多 App 用 AES 加密数据传输,你要找到"钥匙(Key)"就能解密。

Android Studio

一句话:Google 官方的 Android 开发工具(IDE)。 **逆向中的用途:**主要用来**装模拟器**、调试 smali 代码。

APK (Android Package Kit)

一句话:安卓 App 的安装包文件。 **本质:**一个压缩包(zip),改后缀名 `.apk → .zip` 就能解压看内容。 **包含:**代码、图片、配置文件、底层库,全在里面。

apktool

一句话:把 APK 拆开再装回去的瑞士军刀。 **能做什么:** - 解包 APK(变成 smali 代码 + 资源文件) - 修改后重新打包(重打包) **局限:**只能看 smali(不是 Java),不如 jadx 直观。

ARM / ARM64

一句话:手机 CPU 的"指令集"(就像 x86 是电脑 CPU 的指令集)。 **为什么重要:**逆向 `.so` 文件时,看到的汇编代码就是 ARM 指令。 **ARM** = 32 位,**ARM64** = 64 位(现在新手机几乎都是 ARM64)。

ARSC (resources.arsc)

一句话:APK 里存储资源(图片、字符串)ID 的索引表。

ART (Android Runtime)

一句话:Android 系统运行 App 的"引擎"(安卓 5.0 之后取代 Dalvik)。 **对比:**像 Java 的 JVM(虚拟机),但专门跑 Android 的 DEX。

B

Binary Ninja

一句话:一款商业的二进制分析工具,界面好看。对标 IDA Pro。

BlackDex

一句话:一款**脱壳工具**,帮你把加固过的 App 还原成原始 DEX。

C

C / C++

一句话:更底层的编程语言,Android 里一般用来写 `.so` 库(性能敏感、或者藏加密算法)。

CFG (Control Flow Graph)

一句话:控制流图。程序里各个代码块怎么跳转的图。 **哪里见:**IDA / Ghidra 打开函数会看到一张流程图,就是 CFG。

Charles / Fiddler

一句话:抓包神器。能看到 App 和服务端之间在传什么数据。 **类比:**像是"邮件检查员",App 发出去的每一封信都被你拆开看了。

ClassLoader

一句话:Java/Android 用来"加载类"的东西。 **为什么重要:**加固 App 常常动态加载代码,要 Hook ClassLoader 才能看到真实代码。

CrackMe

一句话:故意给你练习破解的小程序。 **类比:**像是"锁匠练习用的锁",可以随便撬。

CTF (Capture The Flag)

一句话:网络安全比赛,有一类题就是逆向 APK 找 flag。 **收益:**练技术 + 拿奖金 + 简历加分。

D

Dalvik

一句话:Android 5.0 之前的运行引擎(现在被 ART 取代了,但"Dalvik 字节码"这个词还在用)。

DEX (Dalvik Executable)

一句话:APK 里的**代码文件**,类似电脑上的 `.exe`。 **所有 Java/Kotlin 代码最终都被编译成 DEX**。一个 APK 可能有多个 dex: `classes.dex`、`classes2.dex` ……

dex2jar

一句话:把 `.dex` 转成 `.jar` (方便用 Java 工具读)。 **现状:**被 jadx 取代了,但老教程还常提到。

Dump(转储)

一句话:把内存里的数据"扒"出来存成文件。 **场景:**App 把 DEX 藏在内存里,你把它 dump 出来就能看代码了。

E

ELF

一句话:Linux/Android 下的可执行文件格式。 `.so` 就是 ELF 格式。

F

FART

一句话:一款**脱壳机**,主动调用每个方法把代码 dump 出来。 **脱壳界的老大。**

Frida

一句话:动态 Hook 神器,**逆向界的瑞士军刀**。 **能做什么:** - App 正在运行时,你能"插入"代码进去 - 打印任意函数的参数、修改返回值 - 绕过 SSL Pinning、绕过各种检测 **为什么要学:**90% 的现代逆向场景都靠它。 **语言:**用 JavaScript 写 Hook 脚本。

Flag

一句话:CTF 比赛里藏在题目中的"答案字符串",找到它就得分。

G

Ghidra

一句话:**免费**的逆向神器,NSA(美国国家安全局)开源的。 **对标:**IDA Pro(收费几万块),Ghidra 完全免费。

Gradle

一句话:Android 开发的构建工具(开发者用,逆向基本不用管)。

H

Hash(哈希)

一句话:把任意数据"压缩"成固定长度的字符串,**不可逆**。 **常见:**MD5(32 位)、SHA1(40 位)、SHA256(64 位)。**逆向场景:**登录密码一般都是 hash 后传输的。

Hook(钩子)

一句话:在程序运行时"钩住"某个函数,让你能改它的行为。 **类比:**邮差本来要把信送到 A,你在半路拦截、看信、改写、再送走。 **工具:**Frida、Xposed。

HTTPS / SSL / TLS

一句话:加密的 HTTP。App 和服务器通信大多用 HTTPS。 **抓包难点:**HTTPS 被加密了,要装"证书"才能抓到明文。

I

IDA Pro

一句话:逆向界的"倚天剑",最强的静态分析工具(**收费,几万块**)。 **能做:**把二进制(.so / .exe)反编译成 C 伪代码。 **F5 键:**按 F5 把汇编变成 C 代码,这一招无人不知。

IDE (Integrated Development Environment)

一句话:集成开发环境,就是个写代码的软件(如 Android Studio、VS Code)。

J

Jadx / jadx-gui

一句话:**最常用的静态分析工具**,把 APK 直接变回 Java 代码。 **为什么好用:** - 免费开源 - 直接拖入 APK,自动反编译 - 能搜字符串、看类、追函数调用 **小白必装**。

Java

一句话:Android 开发的主流语言之一。 **逆向中:**你必须能**读懂** Java 代码(不必会写)。

jarsigner / apksigner

一句话:给 APK 签名的工具。改完 APK 要重新签名才能装。

JEB Pro

一句话:商业的 Android 逆向工具,smali 分析能力顶级。 **收费**。

JNI (Java Native Interface)

一句话:Java 和 C/C++ 之间的"翻译官"。 **为什么存在:**Java 想调用 C 写的代码(通常是为了性能或安全),通过 JNI 实现。 **逆向场景:**很多 App 把加密算法写在 C 里(so 文件),通过 JNI 暴露给 Java 调用。

K

Kotlin

一句话:一门现代语言,Google 2017 年后力推,新 App 基本都用 Kotlin 写。 **逆向中:**Kotlin 编译后还是 DEX, 用 jadx 打开能看到(有些语法稍奇怪)。

Key(密钥)

一句话:加密算法用的"钥匙"。

L

LSposed

一句话:**Xposed 的现代版**,基于 Riru / Zygisk,需要 Root。 **能做:**持久化 Hook(开机就自动注入模块)。

Linux

一句话:Android 的底层就是 Linux 内核。所以 Linux 命令在逆向中常用到。

M

Magisk

一句话:给手机 Root 的神器,且能"隐藏 Root"让 App 检测不到。 **逆向前置:**想玩动态 Hook,手机最好 Root,Root 就用 Magisk。

Manifest (AndroidManifest.xml)

一句话:APK 的"说明书",告诉系统这个 App 有哪些页面、要哪些权限、谁是入口。 **逆向第一步:**打开 APK 先看 Manifest,找到主 Activity(入口)。

MD5

一句话:一种 hash 算法,输出 32 位字符串。 **现状:**已被破解,不安全,但 App 里还很常见。

mitmproxy

一句话:命令行抓包工具,可以用 Python 脚本自动化处理请求。

模拟器

一句话:电脑上假装自己是安卓手机的软件。 **常用:**Android Studio 自带模拟器、雷电、MuMu、夜神、Genymotion。 **坑:**很多 App 会检测模拟器,你学到后面要学"反模拟器检测"。

N

NDK (Native Development Kit)

一句话:Google 给开发者写 C/C++ 代码用的工具包。你学逆向不会直接用,但要知道 so 是 NDK 编译的。

Native 代码

一句话:C/C++ 写的代码,编译后变成 `.so` 文件。 **对比:**Java/Kotlin → DEX, C/C++ → so。

O

Objection

一句话:基于 Frida 的**自动化工具**,一键执行常见 Hook。 **类比:**Frida 是菜刀,Objection 是带菜刀的"料理机器人",很多事它自动干。

OLLVM

一句话:一种代码混淆工具,让你看不懂代码。 **特征:**控制流被搞得像面条,函数变得乱七八糟。 **高阶话题:**有专门"反 OLLVM"的技术。

OkHttp

一句话:安卓 App 最常用的网络请求库。 **逆向中:**Hook OkHttp 就能拦截所有请求,是抓包利器。

P

Packer(加壳器)

一句话:把 APK "包一层壳"保护起来的工具。 **常见加固厂商:**360 加固、腾讯乐固、梆梆、爱加密、阿里聚安全、顶象。 **逆向要做什么:**先"脱壳"才能看到真实代码。

Ptrace

一句话:Linux 系统调用,一个进程可以"附加"到另一个进程上调试它。 **检测场景:**App 会用 ptrace 检测自己是否被调试。

Python

一句话:最常用的脚本语言,逆向脚本 90% 用 Python 写。

R

Root

一句话:获取手机的"最高权限"(管理员权限)。 **为什么逆向需要:**很多 Hook 操作要 Root 才能做。 **工具:**Magisk。

radare2 / Cutter

一句话:免费开源的逆向工具,命令行为主。Cutter 是它的 GUI 版。

S

Shell

一句话:命令行界面。adb shell 就是进入手机的命令行。

Signature(签名)

一句话: - **APK 签名**:证明 APK 是谁发布的,防篡改 - **接口签名**:API 请求带的 `sign` 参数,防止别人乱调接口

Smali

一句话:Android 的"汇编语言",是 DEX 代码的人类可读形式。 **类比:**Java 是中文,Smali 是拼音,DEX 是二进制。 **逆向中:**改代码要改 smali,但读代码可以看 jadx 的 Java 伪代码。

SSL Pinning(证书锁定/固定)

一句话:App 只信任自己预设的证书,即使你装了代理证书也抓不到包。 **破解方法:**Frida Hook 绕过。

SO 文件(.so)

一句话:Android 里的"动态链接库",C/C++ 编译的产物。 **类比:**Windows 的 `.dll`。 **为什么重要:**很多核心算法藏在 so 里,Java 层只是个"调用入口"。

SRC (Security Response Center)

一句话:各大公司的"安全应急响应中心",你找到漏洞提交给他们,会拿奖金。 **比如:**阿里 ASRC、腾讯 TSRC。

T

TracerPid

一句话:Linux `/proc/[pid]/status` 里的一个字段,如果非 0 表示被调试。 **App 反调试**会检查这个值。

U

Unidbg

一句话:用 Java 代码"模拟"执行 so 的工具。 **经典场景:**爬虫想调 App 的签名算法,不用装手机,Unidbg 直接跑 so 拿结果。

V

VMP (Virtual Machine Protection)

一句话:把代码编译成**自定义的虚拟机指令**,让逆向极度困难。 **高级保护手段**,大神才玩得转。

X

Xposed

一句话:老牌 Hook 框架,现在基本被 LSPosed 取代。 **特点:**开机自动加载模块,持久化 Hook。

中文名词

脱壳

一句话:加固 App 的代码被"壳"保护了,"脱壳"就是把真实代码从内存里取出来。

反编译

一句话:把编译后的二进制(如 DEX)**反推**回源代码。jadx 干的就是这事。

反调试

一句话:App 检测自己是否被调试,被调试就崩溃/退出。 **绕过:**Hook 掉检测函数。

反模拟器

一句话:App 检测自己是否跑在模拟器上,是就拒绝运行。

加固 / 加壳

一句话:给 APK 套一层保护壳,防逆向。

爬虫

一句话:自动抓取 App/网站数据的程序。很多人学逆向是为了给 App 写爬虫。

混淆

一句话:把代码里的变量名、类名换成无意义的字符(`a`、`b`、`c` ...),让你看不懂。

签名校验

一句话:App 检查自己有没有被人重新打包,被改了就闪退。 **绕过:**Hook 签名校验函数,让它永远返回"没被改"。

重打包

一句话:反编译 → 改代码 → 再打包成 APK → 装机。 **用处:**去广告、破解 VIP 的经典路线。

下一步

看完辞典,打开 `03_工具手册_每个工具干什么用.md`,了解每个工具怎么用。

学习建议:不用一次性记住所有词,**遇到就查**。查 3 次就记住了。

03 | 工具手册:每个工具干什么用

这份文档告诉你:**每个工具是什么、装在哪、什么时候用、怎么配合 AI 学习**。小白只需要先装 的工具,其他学到对应阶段再装。

工具总览表

工具	类别	免费?	难度	必装?	阶段
jadx	静态	免费	易		入门
apktool	静态	免费	易		入门
Android Studio	环境	免费	中		入门
ADB	环境	免费	易		入门
MT 管理器	静态(手机端)	部分	易		入门
Frida	动态	免费	中		核心
Objection	动态	免费	易		核心
Charles	抓包	付费	易		核心
mitmproxy	抓包	免费	中		核心
LSPosed	动态	免费	中		进阶
IDA Pro	Native	贵	难		进阶
Ghidra	Native	免费	难		进阶
Unidbg	Native	免费	难		进阶
Magisk	Root	免费	中		核心
FART/BlackDex	脱壳	免费	难		高级

一、静态分析工具(看代码用)

1. jadx / jadx-gui (必装)

它是什么: 把 APK 直接反编译成 Java 伪代码的神器。 **小白的第一把武器。**

能干嘛: - 拖入 APK 自动反编译 - 看到结构清晰的 Java 代码(几乎和源代码一样) - 搜字符串、跳转引用、看类继承

装在哪: - GitHub 下载:<https://github.com/skylot/jadx> - 下载 `jadx-gui-xxx.zip`, 解压后双击 `bin/jadx-gui.bat` (Windows)或 `bin/jadx-gui` (Mac/Linux)

怎么用(小白 5 步): 1. 打开 jadx-gui 2. 文件 → 打开 APK 3. 左侧看类,双击打开 4. 按 `Ctrl+Shift+F` 搜索字符串(比如 App 里的提示文字) 5. 右键 → "查看调用"追踪函数

AI 配合:

"这段 Java 代码在做什么?[粘贴代码]" "jadx 打开 APK 后,我在哪里能找到主界面的入口?"

2. apktool (必装)

它是什么: 老牌的 APK 解包工具,能把 APK 拆成 smali + 资源,**还能重新打包**(jadx 不行)。

能干嘛: - 解出 smali 代码(可以修改) - 解出 AndroidManifest(可读的 XML) - 修改后重新打包成 APK

装在哪: - 需要先装 JDK - 下载:<https://apktool.org/>

核心命令:

```
apktool d app.apk          # 解包(d = decode)
apktool b app_folder       # 重打包(b = build)
```

AI 配合:

"apktool 解包后报错 xxx,怎么办?" "我想把 App 里的 'VIP' 文字改成 '免费',在哪个文件里改?"

3. MT 管理器 / NP 管理器

它是什么: **手机上就能用**的 APK 逆向工具,安卓界的"小瑞士军刀"。

能干嘛: - 手机上直接打开 APK, 修改 smali、改图标、改名字 - 傻瓜式签名重打包 - 新手最容易上手的工具

装在哪: MT 管理器官网 / 应用商店(有免费版和 VIP 版)

适合人群: - 只想去广告、改小东西的新手 - 没电脑只有手机的同学

4. Android Studio (环境必装)

它是什么: Google 官方的 Android 开发 IDE。

逆向用途(不用来开发): - 装 **Android 模拟器**(学习前期没手机就靠它) - 配合 smalidea 插件做 smali 断点调试 - 看 AndroidManifest 的结构化视图

装在哪: <https://developer.android.com/studio>

5. Android Killer(老)/ AndroidCrack Tool

它是什么: 国产的 APK 逆向集成工具(图形化), 整合了 apktool、dex2jar、jd-gui。

现状: 被 jadx 基本取代, 但老教程经常提到。新人可以跳过。

二、环境 & 设备工具

6. ADB (必装)

它是什么: Android 调试桥, 电脑和手机对话的命令行工具。

装在哪: - 跟着 Android Studio 一起装(最省事) - 或单独下 Platform-Tools:<https://developer.android.com/tools/releases/platform-tools>

核心命令:

```
adb devices          # 查看连接的设备
adb install app.apk  # 安装 APK
adb uninstall 包名   # 卸载
adb shell            # 进入手机 shell
adb logcat           # 看 App 日志
adb push 电脑文件 手机路径 # 推文件到手机
adb pull 手机文件 电脑路径 # 从手机拉文件
```

AI 配合:

"adb devices 显示 unauthorized 怎么办?" "怎么用 adb 快速看一个 App 的日志?"

7. Magisk (Root 必装)

它是什么: 给手机 Root 的工具,且能"隐藏 Root"让 App 检测不到。

为什么需要 Root: - Frida 在 Root 环境更稳定 - 很多 Hook 操作要 Root 权限 - 才能装 LSPosed

怎么 Root: 1. 解锁 BootLoader(每个品牌不同,搜"[手机型号] 解锁 BL") 2. 刷入 Magisk 官方 zip 3. 装 Magisk App

新手建议: - **别用主力机!** 买个二手 Pixel(Google 亲儿子最好刷机) - 型号推荐:Pixel 3a / 4a / 5(价格 300-1000)

AI 配合:

"我有一台 Pixel 5,怎么刷 Magisk 获取 root?步骤详细点"

8. 模拟器(MuMu / 雷电 / Genymotion)

它是什么: 电脑上跑安卓的软件,没手机时的替代方案。

选哪个: - **MuMu / 雷电:**国产,稳定,有 Root 选项 - **Genymotion:**国外的,干净,付费 - **Android Studio 模拟器:**官方,配置麻烦但可靠

注意: - 很多 App **检测模拟器**,会直接退出 - 学到后期要学"过模拟器检测",或直接用真机

三、动态分析工具(Hook 用)

9. Frida (逆向核心)

它是什么: **动态 Hook 神器**,运行时修改 App 行为。

⚠ 不夸张地说,**学会 Frida 是逆向入门和进阶的分水岭**

能干嘛: - 打印任意函数的参数 - 修改函数返回值(比如让"登录失败"变"登录成功") - 调用 App 里的私有方法
- 绕过证书锁定、Root 检测、反调试

装在哪: 1. 电脑端: `pip install frida-tools` 2. 手机端: 下载对应架构的 `frida-server`, 推到手机运行 - 下载地址: <https://github.com/frida/frida/releases>

核心概念: - 用 **JavaScript** 写 Hook 脚本 - 把脚本注入到目标 App 进程

最简单的 Hook 例子:

```
Java.perform(function() {
    var MainActivity = Java.use("com.example.app.MainActivity");
    MainActivity.checkVip.implementation = function() {
        console.log("VIP 检测被调用");
        return true; // 永远返回 true, 破解 VIP
    };
});
```

AI 配合(最常用):

"帮我写一个 Frida 脚本, Hook com.xxx.App 的 login 方法, 打印参数" "Frida 报错 'Failed to spawn xxx', 怎么解决?" "这段 smali 代码怎么翻译成 Frida Hook 脚本?"

学习资源: - 官方文档: <https://frida.re> - 中文教程推荐: **r0ysue 的 B 站视频**(国内最好的 Frida 讲师)

10. Objection (Frida 的"自动挡")

它是什么: 基于 Frida 封装的**命令行工具**, 很多 Hook 不用写脚本, 一行命令搞定。

装: `pip install objection`

神技:

```
objection -g 包名 explore          # 进入 App
android sslpinning disable         # 一键绕过证书锁定
android root disable               # 一键绕过 Root 检测
android hooking list classes       # 列出所有类
android hooking watch class_method xxx # 监控某个方法
```

适合: - 不想写脚本的场景 - 快速做常规绕过

11. Charles (抓包必装)

它是什么: 最流行的 HTTP/HTTPS 抓包工具,图形化,新手友好。

装在哪: <https://www.charlesproxy.com>(有破解版,付费版 50 美金)

基本配置(抓 HTTPS): 1. Charles 启动 → 开启 SSL Proxying 2. 手机设置代理 → 电脑 IP:8888 3. 浏览器访问 chls.pro/ssl 下载证书 4. 手机装证书(用户证书 + 移到系统证书)

注意: - Android 7+ 默认不信任用户证书,要装到系统级 - 遇到 SSL Pinning 要配合 Frida 绕过

AI 配合:

"Charles 抓不到某个 App 的 HTTPS,显示 unknown,怎么办?"

12. mitmproxy (程序员友好)

它是什么: 命令行抓包工具,用 **Python 脚本自动化**处理请求/响应。

为什么选它: - 免费开源(Charles 收费) - 可以用 Python 写脚本,适合做爬虫/自动化

装: `pip install mitmproxy`

运行: `mitmweb` (有网页界面,比纯终端好用)

13. LSPosed (进阶)

它是什么: 现代版 Xposed,用于**持久化 Hook**(开机自动加载模块)。前置:需要 Root + Magisk。

和 Frida 的区别: - Frida:临时 Hook,App 关闭就失效(适合分析) - LSPosed:开机就加载,永久生效(适合改造 App)

适合: - 写一个去广告模块,自动对所有 App 生效

四、Native 层工具(分析 .so 文件)

14. IDA Pro(收费之王)

它是什么: 逆向界的"倚天剑",**最强的二进制分析工具**。价格:商业版几万块(新手不用买,先学 Ghidra)。

核心能力: - 打开 .so / .exe / .dll - 按 **F5** 把汇编变成 C 伪代码(改变世界的一个快捷键) - 交叉引用、调试、脚本自动化

15. Ghidra (免费替代 IDA)

它是什么: NSA 开源的免费逆向工具,功能接近 IDA Pro。 **强烈推荐小白从这个开始学**,别被 IDA 盗版的法律问题困扰。

装在哪: <https://ghidra-sre.org>(需要 Java)

AI 配合:

"Ghidra 反编译出来这段代码,帮我解释一下在做什么:[粘贴]"

16. Unidbg (爬虫党最爱)

它是什么: 用 Java **模拟执行** .so 文件的框架。 **神奇之处:**不用手机、不用模拟器,直接在电脑上跑 so。

典型场景: App 的 sign 签名算法藏在 so 里,你用 Unidbg 把 so 加载起来,传参调用,就能批量生成签名(爬虫必备)。

装: <https://github.com/zhkl0228/unidbg>(Java 项目,用 IDEA 打开)

17. radare2 / Cutter

它是什么: **免费开源**的逆向工具全家桶。 - **radare2**:命令行(geek 向) - **Cutter**:radare2 的 GUI(推荐新手用这个)

五、脱壳 / 加固对抗工具(进阶)

18. FART

它是什么: **脱壳机**,刷到手机里,能把加固 App 的原始 DEX "挤"出来。 **原理:**在 ART 层 Hook,主动调用每个方法,让代码被加载到内存,然后 dump。

19. BlackDex / FRIDA-DEXDump / Youpk

用途:都是脱壳工具,适用不同场景。 - **BlackDex**:免 Root 脱一些弱加固 - **FRIDA-DEXDump**:基于 Frida 的脱壳 - **Youpk**:全量脱壳机

六、辅助 / 小工具

20. ApkScan-PKID(查壳工具)

拿到 APK **第一步查壳**,看有没有被加固、是哪家的加固。

21. 010 Editor

十六进制编辑器,用来手动改文件头、改二进制。

22. Python + requests + pycryptodome

爬虫三件套,逆向出算法后用 Python 复现。

23. 沉浸式翻译 / DeepL

国外教程都是英文,这俩翻译神器必备。

新手"最小装机清单"(先装这几个就够了)

按这个顺序装:

第 1 周(基础): 1. Android Studio + 模拟器 2. ADB(跟着 AS 一起) 3. jadx-gui 4. apktool

第 3 周(动态): 5. Python 3.10+ 6. Frida(`pip install frida-tools`) 7. Charles 或 mitmproxy

第 6 周(进阶): 8. Ghidra 9. 一台二手 Pixel + Magisk Root 10. LSPosed

第 10 周(高级): 11. Unidbg 12. BlackDex(脱壳)

工具学习通用流程(配合 AI)

对于任何新工具,按这个流程学:

Step 1: 问 AI "XXX 工具是干什么的?一句话说明"
Step 2: 问 AI "在 [我的系统] 上怎么安装 XXX?"
Step 3: 跟着装,报错就把错误粘贴给 AI
Step 4: 问 AI "新手入门 XXX 最常用的 5 个命令/操作是?"
Step 5: 找个 CrackMe App 实战
Step 6: 遇到场景再深入问 AI

下一步

看 [04_AI辅助学习法_怎么跟AI提问.md](#),学会跟 AI 高效对话。

04 | AI 辅助学习法:怎么跟 AI 高效对话

这是最重要的一篇文档。 学会用 AI,你的学习速度能 **快 10 倍**。学不会用 AI,有神仙老师也没用。

一、AI 能给你什么?不能给你什么?

AI 擅长做的

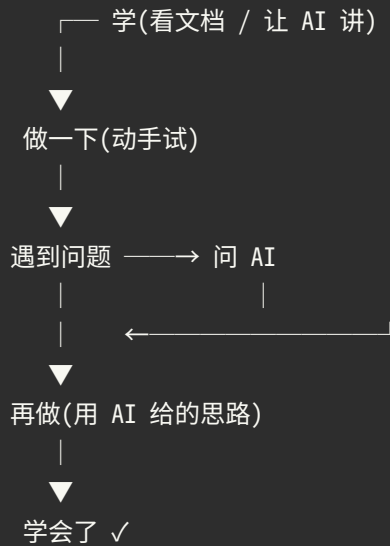
- **解释任何概念**(大白话、带类比)
- **生成代码/脚本**(Frida Hook、Python 解密脚本)
- **调试报错**(粘贴错误信息,90% 能解决)
- **分析代码逻辑**(粘贴一段 smali/Java,让它翻译)
- **总结知识点**(学完一块让它帮你总结)
- **出练习题**(让它给你出题验证学习效果)

AI 不擅长做的

- **替你操作**(安装工具、连手机、点击界面 —— 必须你自己动手)
- **保证绝对正确**(可能有"幻觉",关键结果自己验证)
- **代替思考**(你不动脑,就是"喂答案",学不会)

二、核心心法

心法 1:学 → 做 → 问 → 再做



最致命的错误:看了就算学了。**必须动手。**

心法 2:问具体的,不问抽象的

烂问题:"怎么学逆向?"(太宽,AI 只能给泛泛的答案) 好问题:"我装好了 jadx,想分析一个去广告的 App,具体怎么找到广告相关的代码?"

心法 3:给 AI 足够的上下文

AI 不知道你的: - 操作系统(Win? Mac?) - 手机型号(有没有 Root?) - 现在学到哪一步 - 之前做过什么

每次问,先交代清楚。

心法 4:AI 答不对,换种问法

如果 AI 给的答案不对/听不懂,可以说: - "我是零基础小白,能用更简单的话解释吗?" - "能给个生活类比吗?" - "这个解释里 XX 我不懂,单独展开讲"

三、7 个神级 Prompt 模板(复制即用)

模板 1:概念解释器

你是一位资深的安卓逆向工程师,现在我是零基础小白。

请用大白话解释下面这个概念:

【概念】:_____

要求:

1. 一句话核心定义
2. 一个生活化类比
3. 在 APK 逆向中的具体用途
4. 举一个最简单的例子
5. 常见的相关术语链接

不要用"显然"、"众所周知"这类词。

示例:把 【概念】 填成 JNI ,试试看。

模板 2:代码翻译器

我在逆向一个 Android App,下面是 jadx 反编译的代码:

【代码】:

```
``java  
(粘贴代码)
```

请帮我: 1. 逐行翻译成成人话(每行加注释) 2. 总结这段代码整体在做什么 3. 哪几行是关键逻辑(比如加密、校验、网络请求) 4. 如果我想 Hook 它,应该 Hook 哪个方法、怎么做

模板 3:Frida 脚本生成器

我要 Hook 一个 App,需求如下:

- App 包名:_____
- 目标类名:_____
- 目标方法:_____
- 目的(想看参数 / 改返回值 / 调用它):_____

请帮我: 1. 写一份 Frida Hook 脚本(附中文注释) 2. 告诉我怎么运行它(完整命令) 3. 可能遇到的报错和解决方案

模板 4:报错解决器

我在做 ____ (简述正在做什么)。

我运行的命令:

(命令)

报错信息:

(完整错误)

我的环境: - 操作系统: _ - 工具版本: _ - 目标 App / 设备: ____

我已经尝试过: - _ - _

请帮我分析原因 + 给出 2-3 个解决方案。

> 这是最高频的 Prompt, **保存到记事本!**

模板 5:学习路径规划器

我是零基础,学 APK 逆向。

我的背景: - 会的编程语言: _ (或零基础) - 可用时间: 每天 __ 小时 - 目标(几个月内): _ (比如能给某 App 写爬虫)

请帮我设计一个未来 4 周的学习计划,要求: - 每周目标 - 每天 1-2 小时具体任务 - 每周末的验收标准 - 用什么工具 - 哪些坑要避免

模板 6:知识点验收器

我刚学完 ____ (主题)。

请用以下方式检测我是否真的学会: 1. 出 3 个概念题(让我用自己的话解释) 2. 出 2 个情境题(给我个场景让我说怎么处理) 3. 出 1 个动手题(一个小任务让我实操) 4. 每题后告诉我正确答案和扣分点

模板 7:案例分析器

我想学习一个经典的逆向案例,请帮我设计:

场景:____(比如"去某 App 开屏广告")

请按以下结构输出: 1. 这个案例的**难度**(1-5 星) 2. **需要的前置知识**(列出来) 3. **用到的工具**(列出来) 4. **详细步骤**(每步都告诉我"做什么 + 为什么 + 预期结果") 5. **可能遇到的问题** 6. **做完后能学到什么**

注意:我是小白,步骤要细到每一次点击。

四、"学习循环"工作流(每天这样做)

日常学习循环(推荐)

早上 / 学习前: |—— 问 AI:"今天我要学 XX,给我一份 30 分钟的速成讲解"

学习中: |—— 遇到术语 → 查 02_名词大辞典 or 问 AI |—— 装工具报错 → 用"报错解决器"Prompt |—— 代码看不懂 → 用"代码翻译器"Prompt

学习后: |—— 让 AI 帮你出 3 题 → 用"知识点验收器"Prompt |—— 记录学到的东西(写进自己的笔记)

每周末: |—— 回看本周笔记 |—— 做一个本周的实战小案例 |—— 问 AI:"我这周学了 XX,下周该学什么?"

五、AI 学习中的 5 个反模式(别这么干)

反模式 1:"伸手党"

问:"帮我破解这个 App 的 VIP"

→ AI 不会直接给你"破解成品",会让你自己学过程。

****正确做法****:拆成小问题,一步步问。

反模式 2:只看 AI 的答案,不动手

→ 学的是"看戏",不是"学戏"。

****正确做法****:每个答案后面,****自己跑一遍****。

反模式 3:不验证 AI 的输出

AI 会"幻觉",编造不存在的函数、错误的 API。

****正确做法****:关键代码自己跑一跑,报错就再问。

反模式 4:问题太模糊

"这个为什么不行?" —— AI 不是你肚子里的蛔虫。

****正确做法****:贴命令、贴报错、说环境。

反模式 5:碰到难题就换方向

"Frida 太难了,我不学了学抓包" —— 两周后又说"抓包太难不学了"。

****正确做法****:卡住 30 分钟 → 问 AI → 还不行 → 发帖求助 → 第二天再战。

六、高级用法:让 AI 做你的"陪练"

AI 角色扮演

你现在扮演一位有 10 年经验的安卓逆向面试官。

我在面试初级逆向工程师岗位。请你问我 5 个由浅入深的问题,我回答后你给出评分和改进建议。问题不要太理论,要贴近实战。

开始第一题。

AI 做你的"代码审查员"

这是我写的一个 Frida Hook 脚本,请以资深工程师的视角 review:

(你的脚本)

请指出: 1. 有没有 bug 2. 有没有更优雅的写法 3. 边界情况是否考虑 4. 打分(1-10)并给出提升建议

```
### AI 做你的"阅读加速器"
```

这是我找到的一篇逆向教程,太长了我读不完:【文章链接或内容】_____

请帮我: 1. 用 5 个要点总结核心内容 2. 告诉我最值得记住的 3 个结论 3. 有没有和我学习目标无关的部分可以跳过

```
---
```

```
## 七、配合 Kiro(我)的特殊技巧
```

你现在就在用 Kiro,Kiro 相比其他 AI 有额外能力:

```
### Kiro 能直接操作你的工作区
```

- 帮你**写文件**到文档里(你正在看的这些文档就是这么来的)
- 帮你**跑命令** (装工具、运行脚本)
- 帮你**克隆 GitHub 仓库**

```
### 典型 Kiro Prompt
```

"把今天学到的 Frida 基础知识整理成一个笔记文件, 保存到 APK逆向学习/笔记/ 目录下,格式是 Markdown"

"帮我写一个 Frida Hook 脚本,保存成 hook_login.js, 然后告诉我怎么运行"

"我在做第 05 周的任务 3,帮我检查一下我的进度"

八、 必备工具清单(配合 AI 学习)

工具	用途
Kiro / ChatGPT / Claude	主力 AI 老师
Notion / Obsidian / 飞书文档	记笔记,把 AI 的解释存下来
沉浸式翻译浏览器插件	看英文文档秒变中文
Snipaste 截图	截报错发给 AI
Cursor IDE (可选)	写代码时实时 AI 辅助

九、学习节奏建议

- **每天 1-2 小时**:比每周一次 8 小时更有效
- **每周至少 1 次实战**:哪怕只花 1 小时
- **每月总结一次**:问 AI "帮我出 10 道本月知识综合题"

十、最后的大白话提醒

- > **AI 是"副驾驶",不是"司机"。 **
- > 你握方向盘 → 它给导航提示
- > 你踩油门 → 它提醒你别超速
- > **它永远不会代你开到终点。 **

真正的学习,**发生在你动手的那一刻**。

下一步

打开 `05\_12周学习路线\_每日任务.md`,开始执行!

<div class="page-break"></div>

05 | 12 周学习路线(每周目标 + 每日任务)

- > **按这份计划走,12 周后你能达到"初级逆向工程师"水平。 **
- > 每天 1.5 - 2 小时,每周 1 次实战,全程用 AI 辅助。

总览:12 周 4 阶段

阶段	周次	目标	毕业标志
-----	-----	-----	-----
阶段一:地基	第 1-3 周	编程基础 + Android 基础	能读懂 Java 代码
阶段二:静态	第 4-6 周	会反编译、会改包	能去掉一个 App 的广告
阶段三:动态	第 7-9 周	会 Frida、会抓包	能 Hook 任意函数抓接口
阶段四:进阶	第 10-12 周	会 so 分析 + 对抗	能还原简单签名算法

第 1 周:Java 基础速成 + 环境搭建

本周目标

- 能读懂基础 Java 语法(类、方法、变量、if/for)
- 装好所有基础工具

每日任务

Day 1(1.5h):

- 看 `00\_从零开始` + `01\_知识架构`
- AI 提问:"用 10 分钟教会我 Java 的 5 个核心概念"
- 不需要会写 Java,只要能**读懂**

Day 2(2h):

- 装 JDK 17、Python 3.10、Git
- 装 Android Studio,创建一个模拟器
- AI 辅助每一步(报错直接问)

Day 3(1.5h):

- Java 语法 1:类 / 对象 / 方法 / 字段
- AI 给你写 5 段代码,让你解释每段在做什么

Day 4(1.5h):

- Java 语法 2:if / for / while / switch
- 同上,AI 出题你答

Day 5(1.5h):

- Java 语法 3:继承 / 接口 / 注解
- 重点理解 `@Override`、`implements`

Day 6(2h):

- 读一段真实 Android 代码(让 AI 给你一段简单 Activity 代码)
- 逐行写注释

Day 7(验收):

- AI 出 10 道 Java 代码阅读题
- 自测通过率 > 70% 才能进下一周

本周验收

- [] Android Studio 模拟器能启动
- [] `adb devices` 能看到设备
- [] 能看懂 50 行 Java 代码大概在做什么

第 2 周:Android 基础 + APK 结构

本周目标

- 懂 APK 文件里都有什么
- 懂安卓 App 的四大组件

每日任务

Day 1:APK 结构解剖

- 找一个简单 APK(去官方网站下一个小 App)
- 把它改名为`.zip`解压,看里面有什么
- AI 讲解每个文件/目录的作用

Day 2:AndroidManifest.xml

- 用 Android Studio 打开上面的 Manifest
- AI 教你读懂:`activity`、`permission`、`intent-filter`

Day 3-4:四大组件

- Activity(页面)
- Service(后台服务)
- Broadcast Receiver(广播)
- Content Provider(数据共享)
- 每个让 AI 举一个生活例子

Day 5:APK 打包签名原理

- v1 / v2 / v3 签名区别
- AI 讲解签名机制

Day 6:ADB 深度使用

- 10 个常用命令(AI 给你出题)

Day 7(实战):

- 自己写一个"Hello World"App(AI 手把手教)
- 打包、安装到模拟器

本周验收

- [] 能说出 APK 里主要文件的作用
- [] 会用 ADB 装 / 卸 / 调试 App

第 3 周:Smali 语言基础

本周目标

- 能读懂简单的 smali 代码
- 会小幅修改 smali

每日任务

Day 1:Smali 是什么

- AI 讲解:为什么有 smali、和 Java 什么关系

Day 2-3:Smali 基础语法

- 寄存器(v0, v1, p0, p1)
- 基本指令(const、move、invoke、return)
- AI 让你"翻译":给一段 Java 代码,对照 smali

Day 4:读 if / for 的 smali

- 条件跳转语句

Day 5:读方法调用的 smali

- `invoke-virtual`、`invoke-static`、`invoke-direct` 区别

Day 6:修改 smali 实战

- 找一个 CrackMe App(AI 推荐几个)
- 把 "if 登录成功" 的条件反转

Day 7:重打包

- 用 apktool 解包 → 改 smali → 重打包 → 装机 → 验证

本周验收

- [] 能看懂 smali 里 "调用了哪个方法"
- [] 能成功重打包一次 APK 并在模拟器装上

第 4 周:jadx 深度使用 + 静态分析流程

本周目标

- 精通 jadx 的所有常用功能
- 建立"拿到 APK 该怎么办"的分析流程

每日任务

Day 1:jadx 装好,熟悉界面

- 打开一个 APK 到处点

Day 2:搜字符串定位代码

- 比如 App 登录失败会弹"密码错误",搜这个字符串就能找到代码位置

****Day 3****:追踪函数调用链

- "查找用法"、"跳转到声明"

****Day 4****:看 AndroidManifest 找入口

- 从入口 Activity 一路追到核心逻辑

****Day 5****:导出为 .java 项目

- 方便用 IDE 全局搜索

****Day 6-7(实战)****:

- 随便找一个广告多的小 App
- 用 jadx 定位到广告代码,记录分析过程
- 把分析笔记让 AI review

本周验收

- [] 拿到一个 APK,能在 10 分钟内找到主入口
- [] 能说出一个 App "登录成功后会跳转到哪里"

第 5 周:改包去广告实战

本周目标

- 完整走完一次"去广告"流程

每日任务

****Day 1****:目标选择

- 选一个简单的、有开屏广告的 App(官方下载)
- ****千万别选大厂的****(加固强,新手搞不定)

****Day 2****:定位广告代码

- 搜关键字("广告"、"Ad"、"banner")
- AI 帮你分析哪里是广告入口

****Day 3****:修改策略

- 三种方案:
 1. 让广告方法直接 `return` (最简单)
 2. 把广告视图 `setVisibility(GONE)`
 3. 注释掉广告 SDK 初始化

****Day 4****:修改 smali

- apktool 解包 → 改 → 重打包

****Day 5****:签名 & 安装

- 用 apksigner 签名
- 装机测试

```

**Day 6**:失败 → 调试 → 重做
- 这是最正常的, AI 陪你一起排查

**Day 7**:写笔记总结
- 完整记录过程, 发到看雪论坛或博客

### 本周验收
- [ ] 成功去掉一个 App 的广告
- [ ] 写一篇 500 字的分析笔记

---

## 第 6 周:Python + 基础加密算法

### 本周目标
- 会用 Python 写基础脚本
- 理解 MD5 / AES / RSA / Base64

### 每日任务

**Day 1-2**:Python 基础(如果已会可跳过)
- 变量、字符串、列表、字典、函数
- AI 给 5 个小练习

**Day 3**:哈希算法
- MD5 / SHA256
- Python 调用 `hashlib`

**Day 4**:对称加密
- AES 原理 + Python 实现
- `pycryptodome` 库

**Day 5**:非对称加密
- RSA 原理 + 简单例子

**Day 6**:Base64 / URL 编码
- 不是加密但经常被误以为是

**Day 7**:综合练习
- AI 出题:给一段"加密"字符串, 让你猜是什么算法

### 本周验收
- [ ] 看到一段 32 位字符串能猜出是 MD5
- [ ] 能用 Python 写一个 AES 加解密小工具

---

## 第 7 周:Frida 入门

```


本周目标

- 装好 Frida, 跑通第一个 Hook

每日任务

Day 1: 装 Frida

- 电脑: `pip install frida-tools`
- 手机: 下载对应架构 frida-server, 推到手机
- 遇到问题问 AI

Day 2: 第一个 Hook (Hello World)

- 随便 Hook 一个 App 的方法, 打印调用

Day 3: Hook 参数

- 函数被调用时, 打印它接收的参数

Day 4: Hook 返回值

- 修改返回值 (比如让 `isVip()` 永远返回 true)

Day 5: Hook 构造函数 / 重载方法

- `\$init`、`overload`

Day 6: 调用 App 内部方法

- 用 Frida 直接调 App 的私有函数

Day 7 (实战):

- 找一个带登录的 App
- Hook 登录方法, 打印账号密码 (当然只 Hook 你自己的账号)

本周验收

- [] 成功 Hook 一个方法, 看到参数和返回值
- [] Frida 脚本能独立写一个简单的

第 8 周: 抓包 + SSL Pinning 绕过

本周目标

- 会抓任何 App 的 HTTPS 请求

每日任务

Day 1: 装 Charles (或 mitmproxy)

- 配置手机代理
- 抓 HTTP 请求

Day 2: 装证书抓 HTTPS

- 用户证书 → 系统证书 (Android 7+ 必须)

- Root 手机操作

****Day 3****:遇到 SSL Pinning

- 装证书后还是抓不到 → 说明有证书锁定
- 识别 Pinning

****Day 4-5****:Frida 绕过 SSL Pinning

- 用现成脚本(GitHub 上有 `frida-universal-pinning-bypass`)
- AI 教你改脚本适配新 App

****Day 6****:Objection 一键绕过

- `android sslpinning disable`

****Day 7(实战)****:

- 抓 3 个 App 的登录接口,分析请求参数

本周验收

- [] 能抓到一个中小 App 的完整登录流程
- [] 理解 App 用了什么加密算法传密码

第 9 周:Frida 进阶 + 协议分析

本周目标

- 能分析 App 的接口签名算法

每日任务

****Day 1-2****:Frida 高级 API

- `Java.choose` (遍历内存中的实例)
- `Java.enumerateLoadedClasses`
- `RPC` (Python 调 Frida)

****Day 3****:Hook OkHttp

- 一口气拦截所有请求(很多 App 都用 OkHttp)

****Day 4****:Hook 系统类

- 比如 `MessageDigest` (调 MD5 的时候)
- 能抓到很多"加密之前的原始数据"

****Day 5-6****:签名还原实战

- 找一个 App 的 sign 参数
- 用 Frida Hook 加密函数,拿到输入输出
- 推断算法

****Day 7(实战)****:

- 用 Python 复现签名算法
- 验证:你的 Python 算出来 = App 算出来

```

### 本周验收
- [ ] 能完整分析一个 App 的签名参数生成逻辑
- [ ] 能用 Python 独立生成合法签名

---

## 第 10 周:Native 层逆向入门

### 本周目标
- 能看懂简单的 so 文件

### 每日任务

**Day 1**:JNI 机制
- Java 怎么调 C 代码
- 静态注册 vs 动态注册

**Day 2**:装 Ghidra,熟悉界面
- 打开一个 libc.so 看看

**Day 3**:ARM 汇编速成
- 常用指令 10 个(让 AI 列表)
- 寄存器 R0-R15

**Day 4**:读 so 的函数
- F5 查看反编译的 C 代码

**Day 5**:字符串搜索
- 在 so 里找关键字字符串定位算法

**Day 6**:Frida Hook Native 函数
- `Interceptor.attach(Module.findExportByName(...), {...})`

**Day 7(实战)**:
- 找一个把签名放在 so 里的 App
- 用 Frida Hook JNI 层抓输入输出

### 本周验收
- [ ] 能在 Ghidra 里找到指定函数
- [ ] 能 Hook 一个 Native 函数

---

## 第 11 周:Unidbg 模拟执行

### 本周目标
- 会用 Unidbg 脱机调用 so

```

每日任务

****Day 1****:Unidbg 是什么、为什么

- 装 IDEA(免费社区版)
- clone Unidbg 项目

****Day 2****:跑官方 Demo

- 熟悉工程结构

****Day 3-4****:补环境

- 什么是"环境补全"(很多 so 依赖系统 API)
- 遇到缺的 API 一个个补

****Day 5-6****:调用 so 的签名函数

- 选一个目标 App
- Unidbg 调它的 sign 函数

****Day 7(实战)****:

- Python 通过 HTTP 接口调 Unidbg,批量生成签名

本周验收

- [] 能用 Unidbg 成功调一次 so 函数
- [] 理解脱机跑 so 的意义

第 12 周:加固对抗 + 综合实战

本周目标

- 懂加固原理,会基础脱壳
- 完成一个综合项目

每日任务

****Day 1****:识别加固

- 用 ApkScan-PKID 查壳
- 看 lib 目录判断哪家加固

****Day 2-3****:基础脱壳

- BlackDex / FRIDA-DEXDump 试手

****Day 4****:反反调试概念

- ptrace 检测绕过

****Day 5****:反 Frida 检测绕过

- 改 frida-server 名字、改端口

****Day 6-7(综合实战)****:

- 选一个中等难度的加固 App

- 走完完整流程:查壳 → 脱壳 → 分析 → Hook → 拿结果

本周验收

- [] 成功脱壳一个加固 App
- [] 写一篇完整的综合实战复盘文章

12 周后你具备的能力

- 能独立分析中等难度的 Android App
- 能给 App 写爬虫(接口签名 + 抓数据)
- 能完成 CTF 入门级 Android 题
- 能从广告市场去 App 广告
- 能给 SRC 提交简单漏洞

12 周之后学什么?

方向 1:安全研究员

- 深入漏洞挖掘(WebView 漏洞、组件漏洞)
- 学 AOSP(Android 开源项目)

方向 2:爬虫工程师

- Unidbg 精通
- 对抗各种加固

方向 3:CTF 选手

- 打真实比赛,刷题

方向 4:逆向分析师

- 恶意代码分析
- 病毒样本逆向

执行建议

1. **别跳步**:前面跳了,后面一定补回来
2. **每天打卡**:哪怕只做 30 分钟
3. **卡住先问 AI**,30 分钟还没解决再发帖求助
4. **周末务必做实战**,光学不练等于白学
5. **建一个笔记库**(Notion / Obsidian),把每周的知识沉淀下来

下一步

打开 `06\_实战案例集\_从简单到难.md`, 看有哪些练手项目。

```
<div class="page-break"></div>
```

06 | 实战案例集:从简单到难的 10 个练手项目

> **学逆向,案例比书本重要 100 倍。**

> 下面 10 个案例按难度排列,每个都能让你实打实地掌握一块技能。

难度分级

- 入门(1-2 天)
- 基础(3-5 天)
- 中级(1-2 周)
- 高级(2-4 周)
- 专家(挑战级)

案例 1:Hello CrackMe (入门第一枪)

目标

破解一个故意给你练习的小 App(CrackMe),输入密码就能过关。

需要的技能

- jadx 基本使用
- 会读 Java 代码

步骤

1. 去 GitHub 搜 "android crackme" 下载一个(推荐 OWASP 的 UnCrackable-Level1)
2. 用 jadx 打开
3. 找到密码校验逻辑(搜"密码错误"之类的字符串)
4. 看明文密码,或找到校验函数逻辑后手算

能学到

- 完整跑通一次"拿 APK → 反编译 → 找逻辑"
- 建立逆向的"肌肉记忆"

AI 辅助 Prompt

> "我下载了 UnCrackable-Level1.apk,用 jadx 打开后看到很多类,帮我分析哪个是主入口?"

案例 2:去掉小 App 的开屏广告

目标

找一个有开屏广告的小众 App,把开屏广告去掉。

需要的技能

- jadx + apktool
- Smali 修改
- 重打包 + 签名

步骤

1. 选目标(**避开大厂**,找 3 天内能看完代码的小 App)
2. jadx 打开,搜 "splash" / "ad" / "广告"
3. 定位广告显示的入口
4. apktool 解包
5. 修改 smali:让广告方法直接 return,或改跳转逻辑
6. 重打包 + apksigner 签名
7. 装机验证

常见坑

- 签名不匹配导致装不上 → 先卸载原版
- 改 smali 时改错 `.locals` 寄存器数量 → AI 帮你检查

AI 辅助 Prompt

> "这段 smali 代码是广告的入口,我想让它直接返回,怎么改?注意寄存器数量"

案例 3:修改 App 名称 + 图标

目标

把一个 App 的名字改成你的名字,图标换成你喜欢的。

需要的技能

- apktool 解包
- XML 编辑

步骤

1. apktool 解包
2. 修改 `res/values/strings.xml` 的 `app\_name`
3. 替换 `res/mipmap-\*/ic\_launcher.png`
4. 重打包

能学到

- 资源文件结构
- 基础改包流程

难度

最简单的,作为**练手热身**。

案例 4:Frida Hello World

目标

写出你人生第一个 Frida Hook 脚本,Hook 一个方法并打印参数。

需要的技能

- Frida 环境搭建
- JavaScript 基础(不用深,会改参数就行)

步骤

1. 电脑装 frida-tools,手机跑 frida-server
2. 随便找一个 App(或自己写一个 Demo)
3. 用 jadx 找一个方法名
4. 写 Hook 脚本打印参数 / 返回值
5. `frida -U -f 包名 -l hook.js`

能学到

- Frida 基础工作流
- Hook 的威力

AI 辅助 Prompt

> "帮我写一个最简单的 Frida Hook 脚本,Hook com.test.app 的 Login 类的 check 方法,打印所有参数"

案例 5:Hook 某 App 的登录接口抓密码

目标

找一个**你自己账号**的 App,Hook 登录按钮,抓到明文密码(仅个人学习,不能对他人做!)

需要的技能

- Frida Hook Java 层
- 读 Java 代码能力

步骤

1. jadx 打开 App,找登录 Activity
2. 定位登录按钮点击事件的代码
3. 追到 `login(username, password)` 这种方法
4. Frida Hook 这个方法,打印参数

能学到

- **从静态到动态**的完整分析链路
- 这是**写爬虫**的基础技能

重要提醒 ⚠

只能 Hook 自己的账号,不能对别人做。

案例 6:抓 HTTPS + 绕过 SSL Pinning

目标

找一个大厂 App(比如一个你不介意被研究的小 App),抓到它的 API 请求。

需要的技能

- Charles / mitmproxy
- Frida Universal Pinning Bypass

步骤

1. Charles 配好,装证书到系统
2. 打开 App,发现抓不到 → 有 Pinning
3. 用 Frida 跑通用绕过脚本
4. 成功抓到请求
5. 分析参数:哪些是明文,哪些是签名

能学到

- HTTPS 抓包全流程
- 真实世界的 App 如何保护接口

AI 辅助 Prompt

> "我用 Frida 跑了 frida-universal-pinning-bypass 但还是抓不到,App 会退出,怎么办?"

案例 7:还原一个简单的签名算法

目标

分析一个 App 的 API 签名(sign 参数)是怎么生成的,用 Python 复现。

需要的技能

- 静态 + 动态分析
- Python + 加密库
- 耐心(这是最考验耐心的一步)

步骤

1. 抓包拿到一个请求,看 `sign` 字段
2. jadx 找签名生成的代码(搜 "sign"、"signature")
3. Frida Hook 签名函数,对比输入和输出
4. 分析算法(通常是 MD5(参数排序 + key))
5. 用 Python 实现
6. 测试:你生成的 sign 和 App 生成的一致

能学到

- **爬虫工程师的核心技能**

- 从源码到复现的完整链路

难点

- 参数排序规则
- 盐值(salt)藏在哪儿

案例 8:分析 Native 层(so)的加密算法

目标

App 把关键算法藏在`.so`文件里,你要分析出来。

需要的技能

- Ghidra / IDA
- ARM 汇编基础
- JNI 机制

步骤

1. jadx 发现`native`方法 → 说明函数在 so 里
2. `lib/arm64-v8a/libXXX.so` 用 Ghidra 打开
3. 根据函数名(静态注册)或`RegisterNatives`(动态注册)找到目标
4. F5 反编译看 C 代码
5. Frida Hook 验证分析结果

能学到

- Native 层分析流程
- 这是**高级爬虫**的必备

案例 9:脱壳一个加固 App

目标

找一个用 360 / 梆梆 / 爱加密加固的 App,把真实代码脱出来。

需要的技能

- 脱壳工具(FART / BlackDex)
- Root 环境
- 耐心

步骤

1. 查壳工具识别加固类型
2. 选合适的脱壳方案
3. 跑脱壳机,dump 出 DEX
4. 修复 DEX(某些壳脱出来是"残缺"的)
5. jadx 打开 dump 出的 DEX

难点

- 脱壳机自己也要折腾半天环境
- 不同加固要用不同方案

能学到

- 加固原理
- 对抗思维

案例 10:Unidbg 调 so 做爬虫

目标

不装手机、不开模拟器,直接在服务器上用 Unidbg 调用 App 的签名 so,批量采集数据。

需要的技能

- 所有前面技能的综合
- Java(用 IDEA)

步骤

1. 从 App 里抽出目标 so
2. Unidbg 项目中新建一个类加载 so
3. 疯狂补环境(会报缺这个缺那个)
4. 调用成功,跑通一次签名
5. 封装成 HTTP 接口(比如用 SpringBoot)
6. Python 爬虫请求这个接口拿签名

能学到

- 这是逆向工程师**最值钱的技能**之一
- 一个完整 Unidbg 项目能在爬虫圈卖几万

加餐:CTF 题目练习

推荐平台

- **BUUCTF**:国内 CTF 平台,有大量 Android 题
- **攻防世界**:分类清晰
- **PicoCTF**:国外入门 CTF
- **看雪 CTF**:每年一次,高质量

推荐先做的题

1. BUUCTF - Android - "easy-apk"
2. BUUCTF - Android - "rsa-sign"
3. 看雪论坛的 "新手村"

每个案例的复盘模板

做完每个案例,写一份复盘,格式:

```
``markdown
## 案例名称:_____
- 难度:
- 耗时:__ 小时
- 目标完成度:__%

### 遇到的问题
1. _____
2. _____

### 怎么解决的
1. _____
2. _____

### 用到的工具
- _____

### 学到的新东西
1. _____
2. _____

### 下次可以改进的地方
- _____
```

AI 辅助:复盘写完发给 AI,让它帮你 review 并出 5 道延伸题。

实战通用心法

1. 从小 App 开始,别碰大厂

微信、抖音、淘宝 —— 三个月也搞不定,别浪费时间。

2. 同一个案例做 3 遍

第一遍做完,过一周再做一遍(不看笔记),第三遍写一篇教程给别人。

3. 记录所有报错

建一个"错误库",以后遇到能秒查。

4. 别怕做错

改坏 App 了大不了重装,改错 smali 大不了重来 —— **成本几乎为零**。

5. 加入社区

看雪 / 吾爱破解发帖,遇到问题有人帮。

"打卡墙":做完案例来打勾

- ☐ 案例 1:Hello CrackMe
- ☐ 案例 2:去掉开屏广告
- ☐ 案例 3:改 App 名字图标
- ☐ 案例 4:Frida Hello World
- ☐ 案例 5:Hook 登录接口
- ☐ 案例 6:抓 HTTPS + 绕过 Pinning
- ☐ 案例 7:还原签名算法
- ☐ 案例 8:分析 so 加密
- ☐ 案例 9:脱壳加固 App
- ☐ 案例 10:Unidbg 爬虫

10 个全做完,你就是初级逆向工程师了

下一步

打开 [07_避坑指南_法律与常见问题.md](#),确保你走在正道上。

07 | 避坑指南:法律 & 常见问题 FAQ

这一篇可能是所有文档里最重要的。逆向技术是双刃剑,用好了是专业,用错了是违法。

一、法律红线(务必看完)

绝对不能做的事

行为	法律后果
破解商业软件并发布破解版	《著作权法》《计算机软件保护条例》,可能刑事
逆向金融类 App(银行、支付)并传播	《网络安全法》《刑法》
未经授权入侵他人系统	《刑法》285/286,非法侵入计算机信息系统罪
窃取、倒卖他人数据	《个人信息保护法》《刑法》,数据侵犯罪
制作销售外挂 / 破解工具	《刑法》,提供侵入计算机程序罪
恶意抓取大量数据(爬虫)	《刑法》,非法获取计算机信息系统数据罪
逆向 App 绕过支付	诈骗罪 / 盗窃罪

可以做的事

行为	前提
逆向 自己 开发的 App	无需前提
逆向 开源 App(看 License)	遵守 License 条款
逆向 官方靶场 / CrackMe	它们就是给你练的
分析自己公司授权的 App	有书面授权
做安全研究并通过 SRC 提交漏洞	走官方渠道
分析自己设备上的 App 用于 学习	不传播、不牟利
给自己用的"去广告"修改	不传播、不商用

三大法律文件(了解即可)

1. **《网络安全法》** (2017 年施行)
2. **《数据安全法》** (2021 年施行)
3. **《个人信息保护法》** (2021 年施行)

刑法相关条款: - 第 285 条:非法侵入计算机信息系统罪 - 第 286 条:破坏计算机信息系统罪 - 第 287 条:利用计算机实施犯罪 - 第 253 条之一:侵犯公民个人信息罪

新手保命 3 条军规

1. **只在自己设备、自己账号上做实验**
2. **任何"研究成果"不对外传播、不商用**
3. **发现漏洞走 SRC,不黑产**

二、伦理建议(技术人的底线)

好的使用方向

- 做**安全研究员**,帮厂商发现漏洞
- 做**爬虫工程师**,合规采集公开数据
- 做**CTF 选手**,比赛拿奖
- 做**企业内部安全团队**,保护自家产品

危险的滑坡 ⚠️

一旦走上这些路,很快就会出事: - 帮人破解付费 App → 发布到破解网站 - 给别人写外挂卖钱 - 拿到用户数据转卖 - 给"羊毛党"、"黑产"提供技术支持

朋友们做个普通人,别走歪路。

三、常见问题 FAQ

基础类

Q1:我是零基础文科生,真能学会吗?

A:能,但要更慢一些。 - 前 3 周会很痛苦(Java 语法 + 英文报错) - 度过第 3 周,就是下坡路 - 关键是**动手 + 问 AI**

Q2:需要多长时间能"出师"?

A: - 入门(能抓包 + 简单 Hook):**2 个月** - 进阶(能写爬虫):**6 个月** - 中级(能接商业单):**1 年** - 高级(能拿 CTF 奖):**2-3 年**

Q3:没手机只有电脑,能学吗?

A:能,用模拟器起步。 - 推荐 MuMu / 雷电 - 但**第 7 周学 Frida 时强烈建议买二手 Pixel**(几百块)

Q4:Windows / Mac / Linux 哪个好?

A: - **Mac**:最舒服(Unix 底层,工具兼容好) - **Linux**:工具最全(Ubuntu 推荐) - **Windows**:也能用,需要装 WSL2 模拟 Linux

Q5:英语不好怎么办?

A:不是大问题。 - 装**沉浸式翻译**插件(免费) - 报错粘给 AI 直接翻译 - 术语记熟 100 个就够用

工具类

Q6:模拟器还是真机?

A: - **入门阶段**:模拟器够了 - **Frida 开始**:真机更可靠,模拟器经常被 App 检测 - **最终建议**:二手 Pixel + Magisk 是黄金组合

Q7:jadx 打开 APK 特别卡 / 反编译失败?

A: - 大 APK(> 100MB)确实慢,加内存参数: `jadx-gui -Xmx8g` - 反编译失败可以用 `-r` (只反编译,不解析资源)
- 还是不行?用 **JEB Pro** 或 **IDA Pro**

Q8:Frida 连不上手机?

A:检查清单: 1. `adb devices` 能看到设备吗? 2. frida-server 跑起来了吗?(`adb shell "ps | grep frida"`) 3. 架构对吗?ARM64 手机要用 ARM64 版本的 server 4. 权限对吗? `chmod 755 frida-server` 5. 都不行 → 把报错发给 AI

Q9:Charles 抓不到某 App 的包?

A:常见原因: 1. 没装证书或装的是用户证书(Android 7+ 要系统证书) 2. App 有 SSL Pinning → 需要 Frida 绕过 3. App 用了 HTTP 代理检测 → 会不走代理 4. App 走 WebSocket / QUIC → Charles 抓不了,要用 Wireshark

Q10:IDA Pro 买不起,盗版能用吗?

A: - 盗版在国内灰色地带,但**企业用会吃官司** - 推荐用**Ghidra**(免费开源,NSA 出品) - 学生可申请 IDA 教育版

学习类

Q11:一天学多久合适?

A: - **每天 1-2 小时**比每周 1 次 8 小时有效 10 倍 - 关键在**持续**,别一上来就爆肝三天然后放弃 2 个月

Q12:卡住一个问题好几天怎么办?

A:按顺序尝试: 1. 去 `02_名词大辞典` 看看是不是有术语不懂 2. 问 AI,给足上下文(系统、命令、报错) 3. Google / 看雪论坛搜关键词 4. 发帖求助(看雪论坛 / 吾爱破解) 5. **跳过这个点,先往下学**,回头再看

Q13:AI 有时候答得不对怎么办?

A: - AI 有"幻觉",不要 100% 信 - 关键步骤**自己跑一下验证** - 不对就追问 "你刚才说的 XX 函数,我查文档没找到,能确认一下吗?"

Q14:要不要买课?

A: - **免费资源已经够了**:看雪公开课、B 站、YouTube、官方文档 - 买课的价值在"省时间"和"有人答疑" - 推荐(不是广告):- **r0ysue** 的 Frida 课(国内顶级) - **看雪小程序**(体系化) - **Z0ne Learning**(国外)

Q15:要不要看书?

A:看,但别指望看完。 - 《Android 软件安全权威指南》(非虫)—— **必看**,打基础 - 《逆向工程核心原理》——通用逆向思维 - 看 30% 动手比看 100% 不动手强

职业类

Q16:学完能找什么工作?

A: - **Android 安全工程师**(大厂 30-80 万) - **安全研究员**(看雪、绿盟、奇安信) - **爬虫工程师**(需要逆向能力的都很贵) - **CTF 选手**(上班 + 打比赛拿奖金) - **独立安全研究者**(自由 + 高风险)

Q17:没学历能吃这碗饭吗?

A:能。 - 这行业**重实力不重学历**(比普通程序员还不重视) - 证明方式:**看雪发技术文章** + **CTF 成绩** + **GitHub 作品** - 一篇高质量逆向分析文,比一张本科证好用

Q18:怎么接私活?

A: - 稳妥:通过**看雪认证服务商**接企业单 - 灰色:**知识付费** + **远程 mentor** - 危险:**爬虫私活**(最容易翻车进去)—— **不推荐**

心态类

Q19:我感觉自己笨,学不会怎么办?

A: - 逆向工程师里**没有天才**,都是"熬出来的" - 大佬们也都卡过 3 天不知道某个函数在干啥 - "**卡住**"是**进步信号**,熬过去一次你就升一级

Q20:看到别人大佬 1 天能搞定一个 App,我要一周,正常吗?

A:非常正常。 - 大佬用了 5 年积累经验,你才几个月 - 速度提升来自**案例数量**,你做 100 个案例后速度就上来了 - 别比,**只和昨天的自己比**

四、新手最容易踩的 10 个坑

坑 1:上来就挑战大厂 App

微信、淘宝、抖音有全球最强的加固团队。**先啃小 App**。

坑 2:工具全都装一遍

一开始只装 **jadx + apktool + ADB**。其他用到再装。

坑 3:盗版 IDA 被查到

学生党用 Ghidra,别碰盗版 IDA。

坑 4:Frida 升级后手机端没升级

电脑端 frida-tools 和手机端 frida-server 版本必须一致。

坑 5:改 smali 忘了改 `.locals`

90% 的 smali 改包失败都是这个原因。AI 能帮你检查。

坑 6:重打包后忘了签名

Android 拒绝安装未签名 APK。用 `apksigner sign`。

坑 7:主力机刷机变砖

千万别用自己的主力机刷 Root,买二手学习机。

坑 8:逆向朋友公司 App 被追责

"我没恶意的"不成立。哪怕学习,**也要挑公开可下载的 App**,别逆向内部系统。

坑 9:帮人写爬虫赚钱进去了

爬虫私活 = 高危行业。涉及用户数据或金融数据一律不接。

坑 10:学着学着变成"收藏家"

保存 500GB 视频 = 学会。**动手做 1 个案例 > 看 100 小时视频**。

SOS 五、遇到麻烦怎么办？

技术问题

1. 问 AI(70% 能解决)
2. 看雪论坛搜索 + 发帖
3. 吾爱破解
4. GitHub Issues
5. Stack Overflow

法律问题

- 如果你**已经在做灰产,立刻停止**
- 涉及**金融、个人信息**问题,咨询律师
- 公司内工作,走**安全合规审计**流程

心理问题

- 学习焦虑 → 降低预期,**每天 1 小时坚持**
- 卡住想放弃 → 跳过这个点往下学
- 怀疑自己 → 回头看 1 个月前的笔记,你会感动

六、技术伦理自检清单

开始一个新项目前,问自己:

- ☐ 这个 App 我有合法下载 / 授权吗?
- ☐ 我的研究会不会被用于伤害用户?
- ☐ 我会不会传播未脱敏的数据?
- ☐ 我的"成果"(如脚本、工具)会不会被滥用?
- ☐ 如果被问起,我敢对家人 / 律师解释吗?

全部打勾 → 可以做 任何一项打叉 → 停下来重新评估

七、成为一个"好"的逆向工程师

技术只是工具,关键是用它做什么。

优秀的逆向工程师: - 用技术**保护**用户(发现漏洞提交厂商) - 用技术**创造价值**(合规爬虫、安全产品) - 用技术**教育他人**(写教程、做讲师) - 用技术**应对对抗**(企业反作弊、反爬虫)

糟糕的"技术人": - 破解收钱 - 倒卖数据 - 帮黑产开发 - 用技术恐吓他人

选第一种,路越走越宽;选第二种,早晚进去。

下一步

你已经看完了所有基础文档!

现在回到 **00_从零开始**,按第 5 节的"准备清单"装好环境,然后开始 **05_12周学习路线** 的 Day 1!

祝你学习顺利,未来可期。